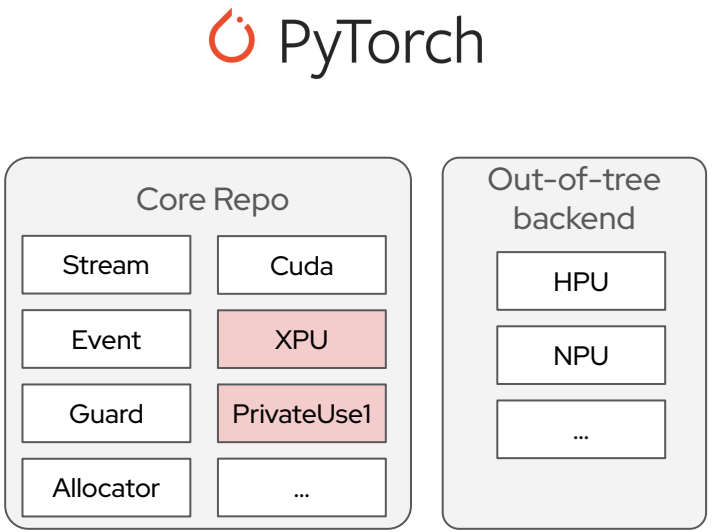
Challenges and Solutions in the Development of PyTorch Ecosystem Accelerators

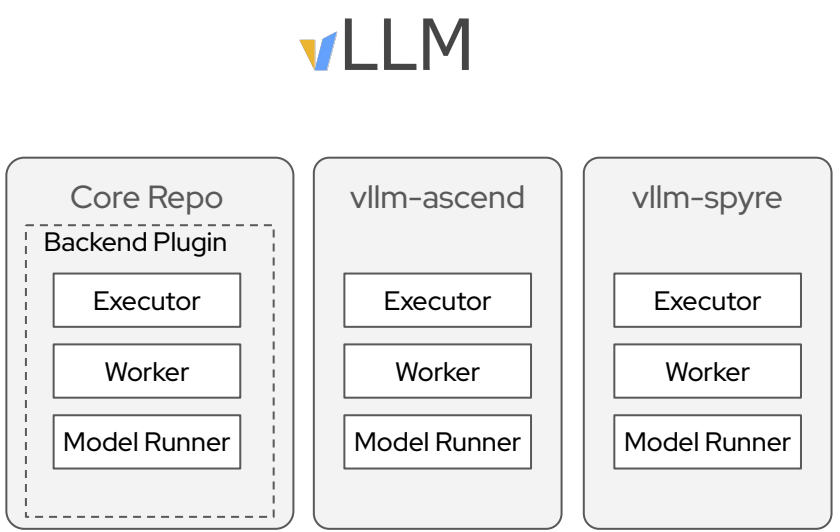
Zesheng Zong (Huawei), Chuanqi Wang (Intel), Jerome Anand* (Intel)

Agenda

- Challenges Faced by Accelerator Developers
- Solution and Enhancement via Exploratory Group
- Example of CI Infra for Accelerators work with Gaudi

Accelerator Integration





Challenges Faced by Backend Developers

Version Compatibility: BC-breaks not aware for accelerators in ecosystem.

Lack of Quality Standards: Hard to verify the level of support operation of new accelerators.

Compromised User Experience: Users may encounter unexpected issues when using less common accelerators.

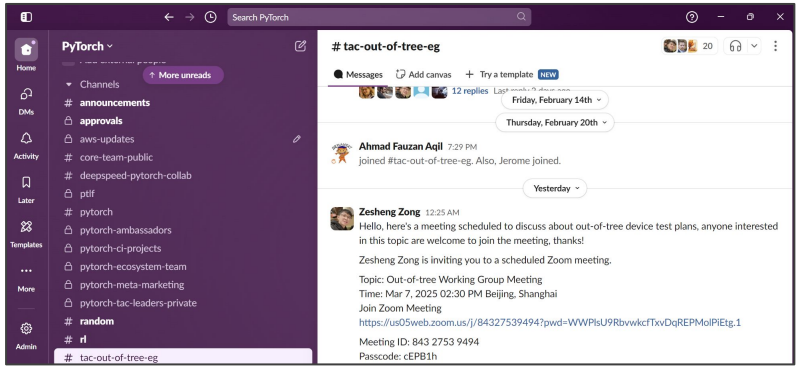
Low Efficiency Development: Poor guidance for backend developers .

Issues of Integration

Type	Description	Occurr Counts	Example PR
Refactoring	Turn Allocator::allocate into non-const, derived class' override function is not modified.	1	#120969
Refactoring	Use DeviceIndex instead of int in CUDA wrappers, derived class' override function is not modified.	1	#119142
Refactoring	Move new trace utils from source to header, which leads to some symbols can't be found.	1	#114367
Refactoring	Migrate to getCvar* functions for env variable checking, which leads to function name can't be found.	1	#113797
New Features	Add support for new data types, data type assert fails.	3	#107586, #116594
New Features	Add function to materialize COW storages, which add a pure virtual function Allocator::copy_data, derived class didn't implement this pure virtual function.	2	#117053, #113396
Refactoring	Make macro with AMP more generic.	1	#124050

Exploratory Group for Accelerators in Ecosystem

PyTorch Slack #tac-out-of-tree-eg



2024.12	Propose: Build quality validations and standards for out-of-tree accelerators.
2025.01	Create exploratory group for accelerators in PyTorch Ecosystem.
2025.03	Upstream repo (pytorch-fdn/oota) for collaboration
2025.06	CI Infra refactor to improve extensibility and more enhancement for accelerators.
Future	Quality Standards for accelerators in PyTorch Community.

Exploratory Group for Accelerators in Ecosystem

Goal

To support diverse AI training hardware within the PyTorch community by reducing integration complexity, ensuring consistent quality and reliability, thereby fostering a vibrant and scalable PyTorch ecosystem for a variety of hardware backends.

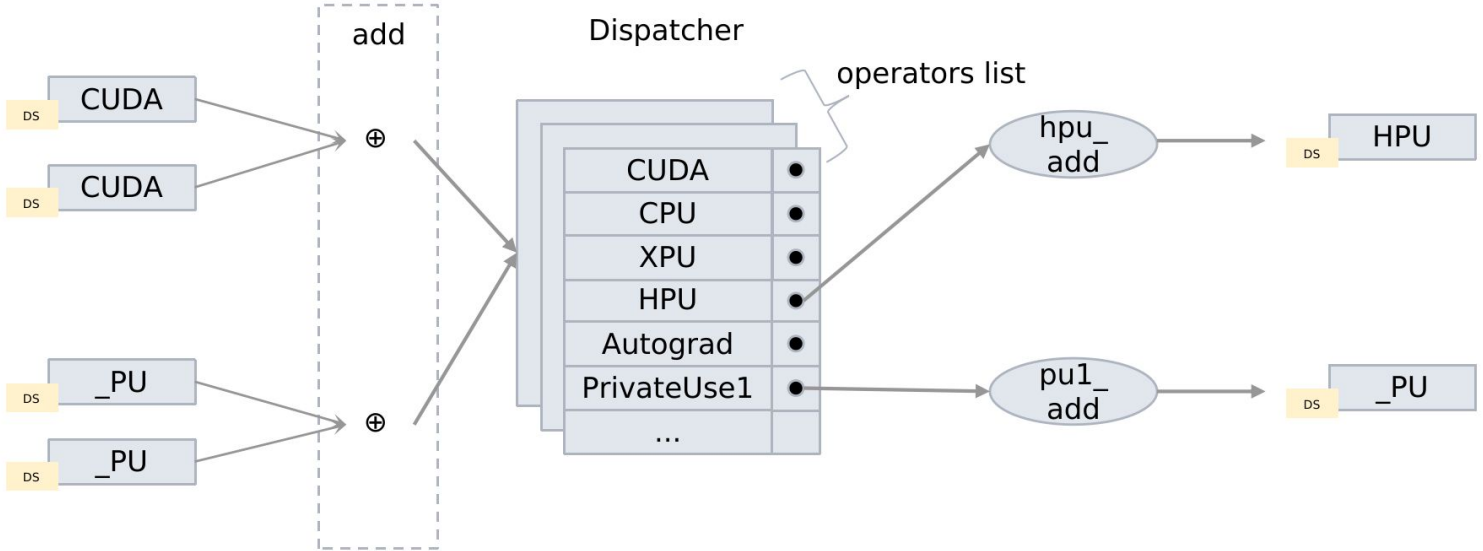
Achievements

- Reduce complexity
  - More device-generic APIs and migrate code of ecosystem tools
  - Enhance PrivateUse1 mechanism
- Quality Assurance
  - Backend specific UT's with extensible CI ensuring sanity with PT core
  - Out-of-tree Accelerators Test Infra
- Promote Hardware Ecosystem
  - Promote via community blogs
  - Offline meetup

PyTorch Ecosystem Accelerator Integration

PrivateUse1 integration mechanism

Match the robustness and versatility of established keys like CUDA and CPU,it involved significant updates to core components such as **AMP (Automatic Mixed Precision), Autograd, Distributed Training, Checkpointing, DataLoader, Optimization,** and **Quantization**, etc.



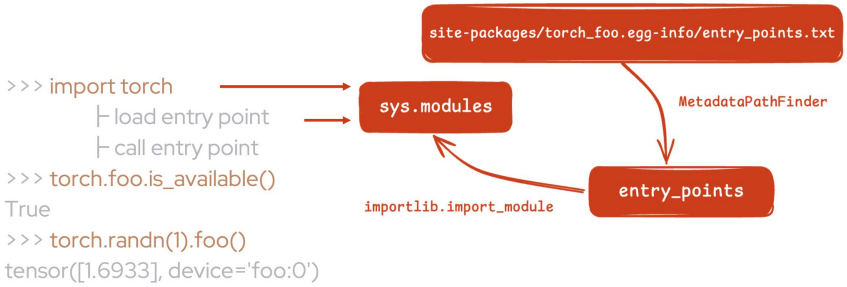
Autoload Accelerator Extension

The extension autoloading mechanism enables PyTorch to automatically load out-of-tree backend extensions without explicit import statements.

- [RFC] Autoload Device Extension #122468
- PR #127074

PyTorch 2.5 Release

- [Prototype] Autoload Device Extension




```
setup(
    name="torch_npu",
    version="2.5",
+   entry_points={
+       'torch.backends': [
+           'torch_npu = torch_npu:_autoload',
+       ],
+   }
)
```

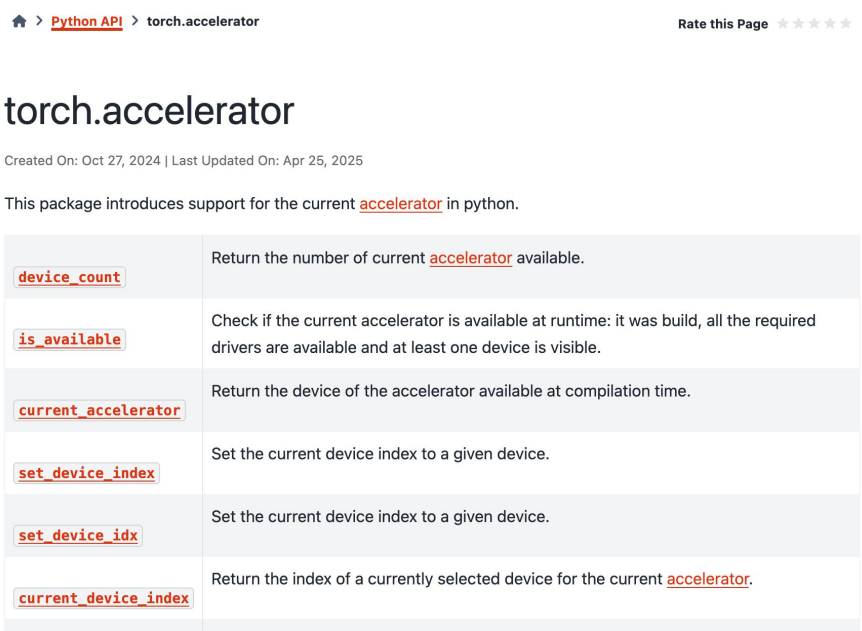
A device-agnostic Python Runtime API

[RFC] A device-agnostic Python runtime API design for stream-based accelerators #128403

PyTorch Roadmap 2025H1

[P0] Build user-facing device-generic API:

- Add more APIs to the torch.accelerator namespace.
- Deliverable: migrate 1 hf repo, ao and torchtune to use this API with 90% reduction of cuda-specific calls there.



Python API > torch.accelerator

Rate this Page

torch.accelerator

Created On: Oct 27, 2024 | Last Updated On: Apr 25, 2025

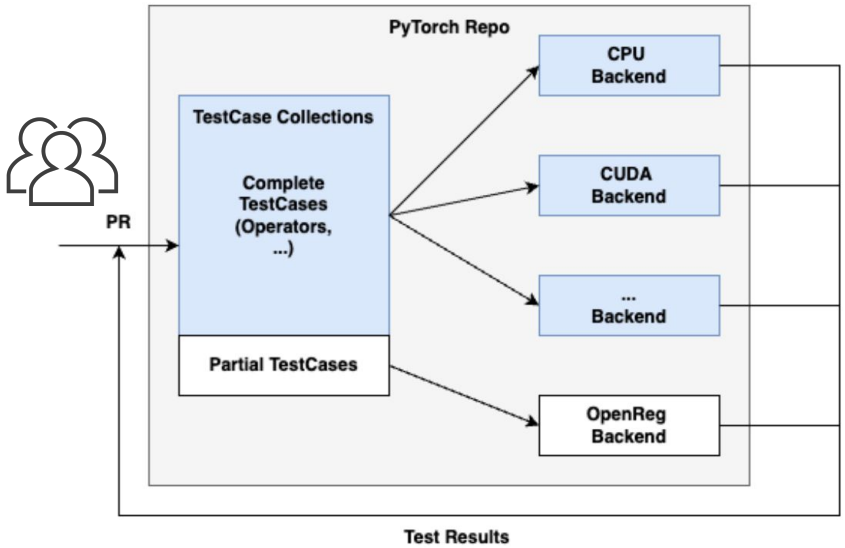
This package introduces support for the current accelerator in python.

`device_count`	Return the number of current accelerator available.
`is_available`	Check if the current accelerator is available at runtime: it was build, all the required drivers are available and at least one device is visible.
`current_accelerator`	Return the device of the accelerator available at compilation time.
`set_device_index`	Set the current device index to a given device.
`set_device_idx`	Set the current device index to a given device.
`current_device_index`	Return the index of a currently selected device for the current accelerator.

OpenReg: A CPU-based Backend Simulation

OpenReg provides a mechanism to do fast change verification based on the existing PyTorch CI/CD workflow:

- Lightweight mock for Running tests in upstream CI/CD.
- The full-function integration of the new backend based on the third-party device integration mechanism.

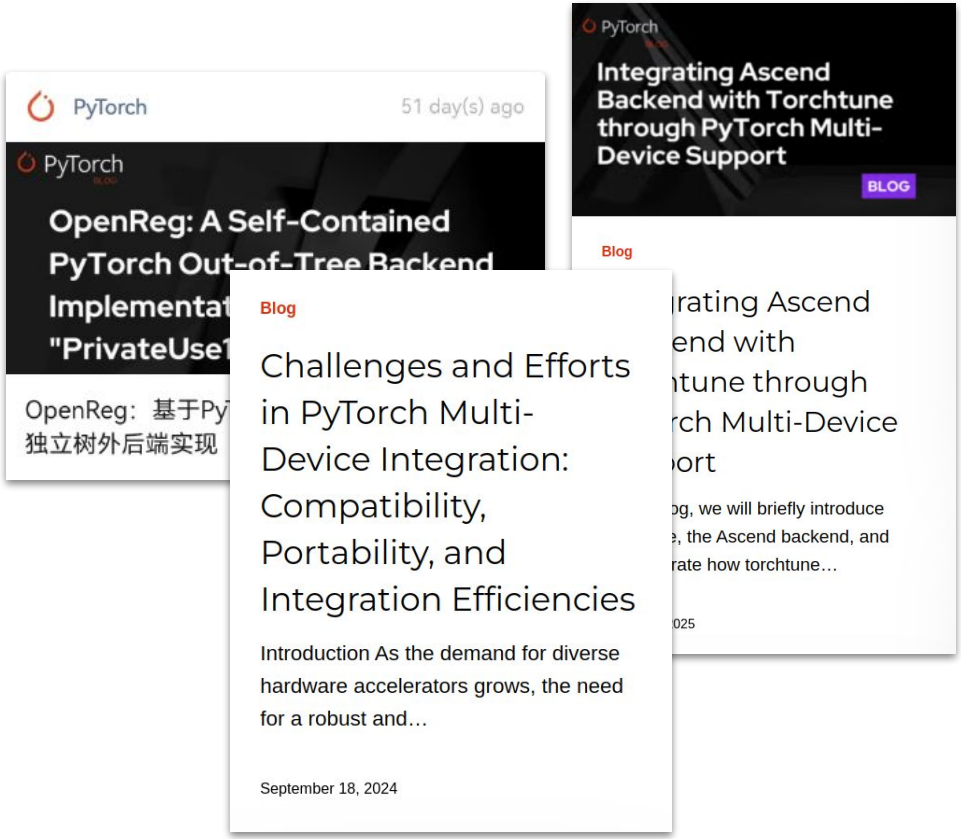


Ecosystem Accelerator Promotion

WeChat 知乎

PyTorch

bilibili



CI Test Infra workflow with Gaudi

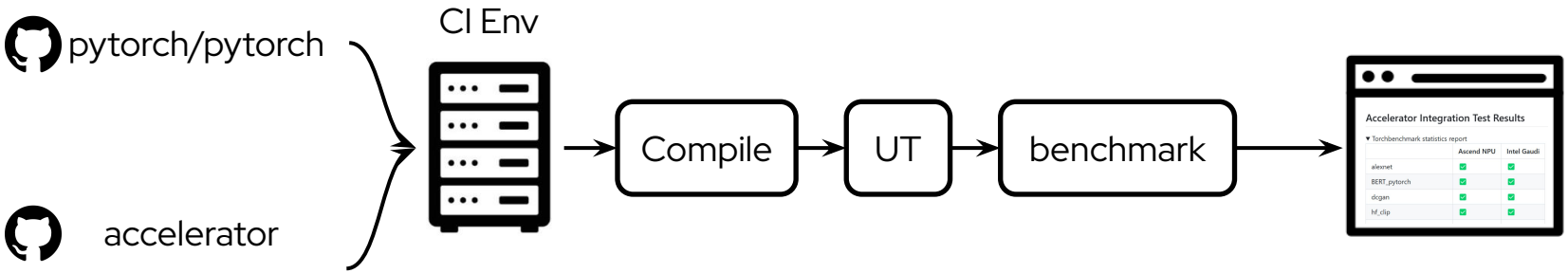
Goal of CI Test Infra

Building an evolving quality validations and standards for PyTorch out-of-tree accelerators.

- Integration Tests
- E2E Model Validations
- Accelerator Quality Standards *(in Future, RFC)*

More details in Validation of Out-of-Tree Accelerator Integration for PyTorch

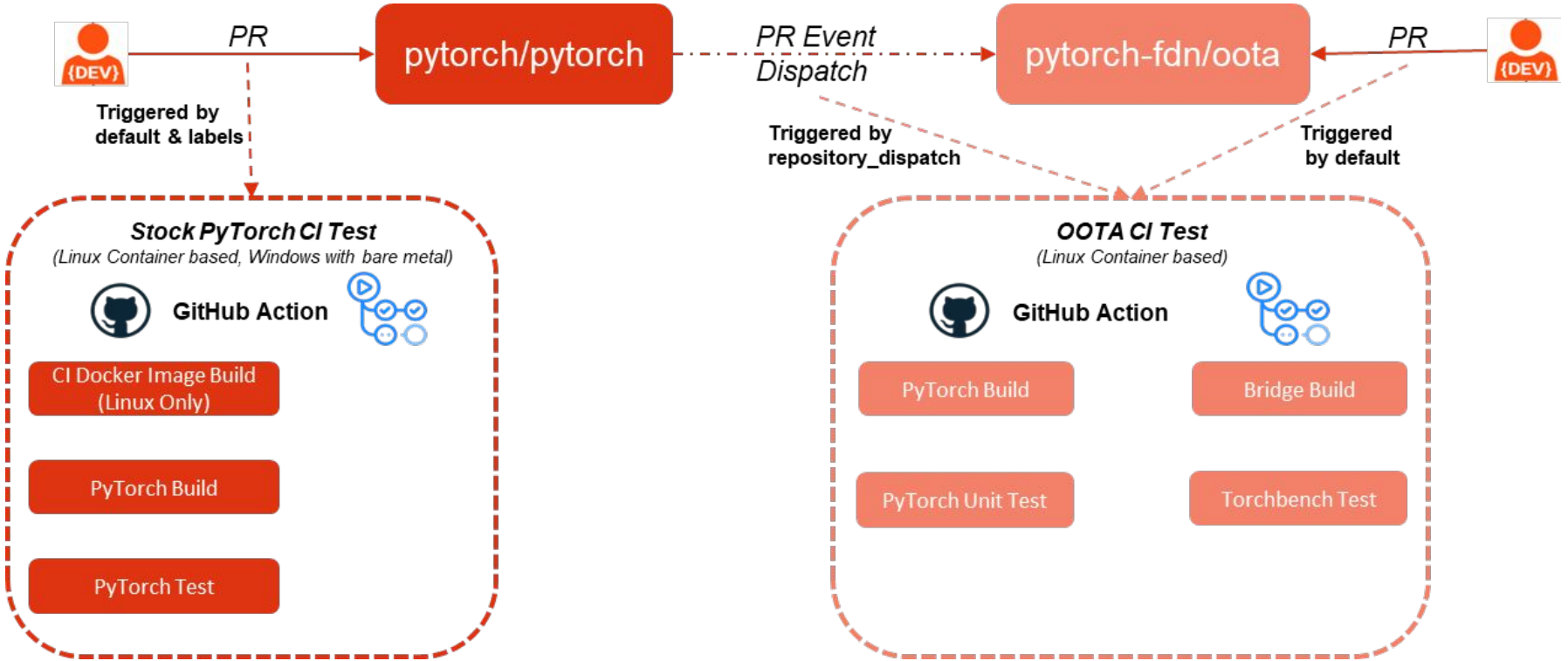
Currently what we have



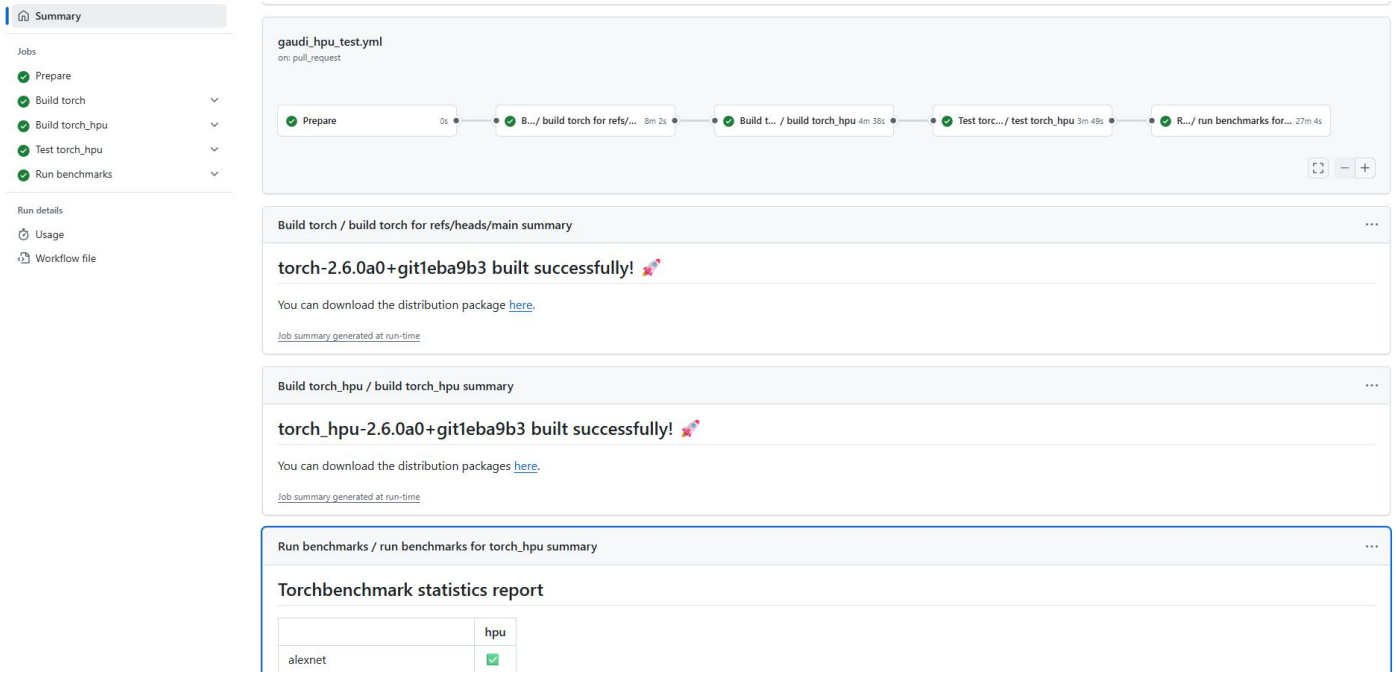
1. Workflow to Build & Test (Periodic)
2. Torch Benchmark Integration
3. Running Result Display
4. Docs for Newcomer

Repo: pytorch-fdn/oota: out-of-tree-accelerators repository

Github Actions workflow snippet



OOTA + Intel Gaudi + TorchBench



How to enable the CI test Infra for a new accelerator?

Step 1	Step 2	Step 3	Step 4
Open Source Customize the dispatch event in PyTorch repo (Optional)	Register new self-hosted runners for the new accelerator	Add workflows with test trigger entrance and standard test scope enabling in OOTA repo	Show the test results on the Dashboard
Closed Source	Redispatch the event from OOTA repo to the accelerator bridge private repo	Enable the test trigger entrance and standard test scope in private accelerator bridge repo	

Thank you